HERMES DESKTOP

Guía completa de instalación, configuración y potenciación en Windows

OpenRouter · modelos gratuitos · modelos locales · memoria · skills · bots · MCP · mensajería · cron · computer use

Objetivo: tener un agente personal potente ejecutándose en tu propio PC, sin depender de una VPS para empezar.

Investigado a partir de la documentación oficial de Hermes Agent / Nous Research y de la documentación oficial de OpenRouter. Verificación de fuentes: septiembre de 2026.

Índice

#	Sección
1	Qué es Hermes Desktop y por qué usarlo en local
2	Instalación en Windows mediante el instalador gráfico
3	Primer arranque y configuración inicial
4	Conectar OpenRouter y obtener la API key
5	Elegir modelos: gratis, premium y routing
6	Usar modelos locales con Ollama / LM Studio
7	Entender SOUL.md, USER.md, MEMORY.md y AGENTS.md
8	Cómo funciona el aprendizaje y la creación de skills
9	Bots: crear agentes especializados
10	Skills, plugins y MCP: ampliar las capacidades
11	Mensajería: Telegram, Discord, Slack, WhatsApp y más
12	Cron: tareas recurrentes y agentes programados
13	Computer Use, navegador, terminal y archivos
14	Configuración avanzada para exprimir Hermes
15	Seguridad, límites y buenas prácticas
16	Recetas prácticas y setup recomendado
17	Fuentes oficiales y enlaces

1. Qué es Hermes Desktop

Hermes Agent es un agente autónomo open source de Nous Research. Hermes Desktop no es un chatbot separado: es la interfaz gráfica del mismo agente que puede ejecutarse por CLI y gateway. La configuración, las claves, las sesiones, las skills y la memoria son compartidas entre las distintas superficies.

Esto es importante porque permite empezar en un PC Windows y, si más adelante quieres pasar a una VPS, servidor o GPU, no tienes que reinventar el agente desde cero. Puedes conservar el concepto de perfil, memoria, skills y configuración.

Idea clave: el ordenador local puede ser tu primera infraestructura. No necesitas alquilar una VPS para experimentar con un agente que puede leer/escribir archivos, ejecutar comandos, usar navegador, conectarse a MCP, programar tareas y recibir mensajes.

Qué aporta Desktop frente a la terminal

- Chat visual con streaming y actividad de herramientas en tiempo real.
- Explorador de archivos y previsualización de resultados dentro de la aplicación.
- Gestión gráfica de proveedores, modelos, herramientas, MCP y credenciales.
- Sesiones múltiples, pestañas y ventanas independientes.
- Artifacts: galería de archivos, imágenes y enlaces generados por sesiones.
- Quick Entry mediante atajo global para enviar un prompt sin abrir la ventana principal.
- Voice y otras superficies del mismo agente.
- Bot Mode para crear agentes especializados con su propio perfil.

Estado actual: Hermes Desktop está disponible para Windows 10/11, macOS y Linux. La web oficial indica que Desktop es gratuito y de código abierto bajo MIT; el modelo y determinados servicios externos sí pueden tener costes.

Página oficial de Desktop
https://hermes-agent.nousresearch.com/desktop
Documentación oficial de Desktop
https://hermes-agent.nousresearch.com/docs/user-guide/desktop

2. Instalación en Windows con el instalador gráfico

La ruta recomendada para alguien que quiere evitar la terminal es descargar el instalador de Hermes Desktop desde la página oficial y ejecutar el .exe. En el primer arranque, Desktop puede provisionar el runtime de Hermes y sus dependencias.

Paso 1 — descargar

- Abre la página oficial de Hermes Desktop.
- En Windows selecciona «Download desktop app».
- Descarga el instalador oficial .exe y ejecútalo.
- Si Windows Defender / SmartScreen muestra un aviso, verifica que el instalador procede del sitio oficial antes de continuar.

Descarga oficial de Hermes Desktop
https://hermes-agent.nousresearch.com/desktop

Paso 2 — instalación

- Acepta la instalación y deja que el instalador configure el runtime.
- No necesitas instalar manualmente Python, Node, ripgrep o ffmpeg para el camino normal: el bootstrap de Hermes gestiona las dependencias necesarias.
- En Windows, el entorno nativo utiliza un directorio de datos bajo %LOCALAPPDATA%\hermes. La ubicación puede cambiar si defines HERMES_HOME.
- Tras la instalación, abre Hermes Desktop desde el menú Inicio.

Qué ocurre detrás de la interfaz

La aplicación Desktop usa un backend de Hermes en segundo plano. La documentación describe una arquitectura en la que Electron proporciona la aplicación nativa y Hermes Agent se ejecuta como proceso headless, exponiendo la interfaz de agente que consume Desktop. Por eso Desktop y CLI pueden compartir el mismo estado.

Dependencias que Hermes puede gestionar en Windows incluyen Python, Node.js, Git/PortableGit, ripgrep y ffmpeg; algunas herramientas añaden dependencias adicionales cuando se utilizan por primera vez, por ejemplo el stack de navegador.

Consejo: si una instalación falla durante la fase de descarga/compilación, no reinstales diez veces. Comprueba primero conectividad a GitHub/npm, proxy/firewall y el log del instalador. La documentación de Desktop contempla incluso reintentos y mirrors para descargas de Electron.

Alternativa si prefieres PowerShell

Instalación nativa de Hermes en Windows

```
iex (irm https://hermes-agent.nousresearch.com/install.ps1)
```

Este método instala el CLI y después puedes lanzar Desktop con hermes desktop. No es necesario si ya vas a utilizar el instalador gráfico.

3. Primer arranque y configuración inicial

La primera configuración debe hacerse en capas. La recomendación de la propia documentación es conseguir primero una conversación normal que funcione y después añadir gateway, cron, skills, voz, routing y otras capacidades.

Checklist de primer arranque

Paso	Qué configurar	Resultado
1	Proveedor de inferencia	Hermes sabe qué API/modelo utilizar.
2	Modelo principal	Tienes un modelo con suficiente contexto y tool calling.
3	Herramientas	Activar solo las que realmente necesitas.
4	Directorio de trabajo	Hermes sabe en qué proyecto operar.
5	SOUL / USER / MEMORY	Identidad y memoria coherentes.
6	Skills	Workflows reutilizables disponibles.
7	Bots	Especialistas separados cuando tenga sentido.
8	Cron / Messaging	Automatización y acceso remoto opcionales.

En Desktop, Settings concentra gran parte de esta configuración y evita editar YAML para las operaciones habituales. Aun así, conocer los archivos es muy útil para hacer backups, versionar proyectos y depurar.

Un requisito especialmente importante: Hermes requiere un modelo con al menos 64K tokens de contexto. Para modelos locales, configura también el servidor local con una ventana de contexto de al menos 64K.

Primer test recomendado

Prompt de verificación

```
Lee el directorio de trabajo actual, identifica el tipo de proyecto,
resume su estructura en 10 puntos y dime qué herramientas tienes
disponibles. No modifiques ningún archivo todavía.
```

Si este test funciona, entonces añade herramientas de escritura, navegador, MCP o computer-use una a una. Así, si algo rompe el flujo, sabrás qué componente lo causó.

4. Conectar OpenRouter y obtener la API key

OpenRouter funciona como una capa de acceso a múltiples modelos mediante una API compatible. Para Hermes es especialmente útil porque puedes cambiar de familia de modelos sin rediseñar el agente y, además, consultar modelos gratuitos disponibles en ese momento.

Crear la API key

- Crea/inicia sesión en OpenRouter.
- Abre la sección de API Keys.
- Crea una nueva key y ponle un nombre identificable, por ejemplo «Hermes Desktop» o «Hermes-PC».Establece, cuando corresponda, límites de gasto y/o expiración. OpenRouter permite límites y fechas de expiración para las claves.
- Copia la key inmediatamente: OpenRouter documenta que el valor en texto plano se muestra solo en la creación y no puede recuperarse después.

OpenRouter — crear API key
https://openrouter.ai/docs/api/api-reference/api-keys/create-keys
OpenRouter — claves
https://openrouter.ai/keys

Conectar la key a Hermes

La forma más cómoda es hacerlo desde Settings → Providers / API Keys en Desktop. Si trabajas desde CLI, Hermes separa secretos de configuración: las claves se guardan en el .env de Hermes, mientras que los parámetros no secretos viven en config.yaml.

Variable principal

```
OPENROUTER_API_KEY=sk-or-v1-...
```

Regla de seguridad: nunca publiques la API key en un tutorial, commit, captura de pantalla, SOUL.md, USER.md, MEMORY.md o skill. Trátala como una contraseña.

Verificación

Desde CLI, si ya tienes Hermes instalado

```
hermes model
```

Selecciona OpenRouter y después el modelo. Si prefieres no usar CLI, haz la selección desde el picker de modelos de Desktop.

5. Elegir modelos: gratis, premium y routing

No existe un único «mejor modelo» para Hermes. Un agente realiza tareas distintas: razonamiento, código, visión, compresión de contexto, extracción web, títulos, etc. Hermes permite asignar modelos auxiliares por tarea, lo que abre una optimización importante de coste/rendimiento.

Modelos gratuitos de OpenRouter

OpenRouter mantiene una colección dinámica de modelos gratuitos. La opción openrouter/free actúa como router y selecciona un modelo gratuito disponible que sea compatible con los requisitos de la petición, por ejemplo tool calling, visión o structured outputs. La disponibilidad de modelos gratuitos cambia con el tiempo.

OpenRouter — Free Models
https://openrouter.ai/collections/free-models
OpenRouter — Free Models Router
https://openrouter.ai/docs/cookbook/get-started/free-models-router-playground

Para un agente, no conviene asumir que «gratis» equivale a «ideal». Comprueba contexto, tool calling, velocidad, límites y calidad de razonamiento. El catálogo gratuito puede cambiar.

Estrategia recomendada de modelos

Tarea	Estrategia
Agente principal	Modelo con buen tool calling, razonamiento y ≥64K de contexto.
Visión	Modelo multimodal específico si el principal no es bueno en imágenes.
Compresión	Modelo rápido/barato con contexto suficiente.
Extracción web	Modelo rápido; evita gastar un modelo premium en tareas mecánicas.
Jobs recurrentes	Modelo barato por defecto y uno premium solo para tareas críticas.
Bots especializados	Pin de modelo por Bot según su objetivo.

Hermes permite configurar modelos auxiliares y cadenas de fallback. Un ejemplo práctico es usar un modelo premium como cerebro principal y un modelo económico/gratuito para tareas auxiliares.

OpenRouter también ofrece routers; para empezar, sin embargo, es mejor elegir un modelo explícito, medir el comportamiento y después experimentar con routing.

6. Modelos locales: Ollama y LM Studio

Si tienes GPU y quieres que la inferencia ocurra en tu propio equipo, Hermes puede conectarse a endpoints OpenAI-compatible locales. La documentación menciona expresamente Ollama, LM Studio, vLLM y llama.cpp/custom endpoints.

La arquitectura queda así:

Hermes Desktop → servidor local → modelo local → herramientas de Hermes.

Esto significa que el modelo puede estar completamente local mientras Hermes conserva sus capacidades de agente. Sin embargo, las herramientas web, algunos servicios externos, mensajería o MCP remoto pueden seguir generando tráfico fuera del equipo.

Ollama

- Instala Ollama y descarga un modelo con capacidad de tool calling adecuada.
- Configura una ventana de contexto de al menos 64K para Hermes.
- En Hermes selecciona el proveedor local correspondiente o un custom endpoint OpenAI-compatible.
- Empieza con tareas pequeñas y mide memoria/VRAM antes de delegar workflows complejos.

LM Studio

- Levanta el servidor local OpenAI-compatible de LM Studio.
- Selecciona el proveedor LM Studio en Hermes o usa un custom endpoint.
- Asegúrate de que el contexto configurado en el servidor sea ≥64K.
- Comprueba que el modelo exponga tool calling de forma compatible.

Regla práctica de hardware

Los modelos locales grandes consumen mucha VRAM/RAM y el rendimiento depende de cuantización, tamaño del modelo, contexto y backend. Para un agente, una respuesta rápida y fiable con herramientas suele ser más útil que ejecutar el modelo más grande que tu PC pueda cargar lentamente.

Configuración conceptual de endpoint

Ejemplo conceptual de custom endpoint

```
provider: custom
base_url: http://127.0.0.1:11434/v1
model: <modelo-local-compatible>
```

La sintaxis exacta depende del proveedor y de la versión de Hermes; usa el picker de modelos de Desktop o hermes model para evitar errores de configuración.

7. SOUL.md, USER.md, MEMORY.md y AGENTS.md

Estos archivos no son cuatro versiones de la misma memoria. Tienen funciones diferentes y, según la documentación actual, incluso tienen propietarios y momentos de carga distintos.

Archivo	Función	Quién lo mantiene	Dónde
SOUL.md	Identidad, personalidad, tono, principios y estilo del agente.	Tú; Hermes crea una plantilla inicial si no existe.	HERMES_HOME/SOUL.md
USER.md	Perfil del usuario: preferencias, estilo, expectativas, contexto estable.	Hermes mediante la herramienta de memoria.	HERMES_HOME/memories/USER.md
MEMORY.md	Notas personales del agente: entorno, convenciones, aprendizajes y detalles útiles.	Hermes mediante la herramienta de memoria.	HERMES_HOME/memories/MEMORY.md
AGENTS.md	Instrucciones del proyecto: arquitectura, comandos, convenciones y workflows.	Tú / el proyecto.	Directorio del proyecto y subdirectorios

Corrección importante respecto a la simplificación habitual

Es común decir «MEMORY.md guarda lo que Hermes ha aprendido» y «USER.md define quién eres tú». Es una buena explicación para empezar, pero técnicamente ambos forman parte de la memoria persistente y se inyectan como una instantánea al inicio de la sesión. La documentación actual sitúa USER.md y MEMORY.md dentro de HERMES_HOME/memories/, no necesariamente junto a SOUL.md.

Además, AGENTS.md no es una memoria global: es contexto de proyecto. Hermes puede descubrir copias anidadas conforme navega por subdirectorios. Existe también .hermes.md / HERMES.md para instrucciones específicas de Hermes con prioridad sobre AGENTS.md.

Ejemplo de SOUL.md

Identidad global del agente

```
# SOUL.md

Eres mi agente técnico personal.
Prioriza precisión, trazabilidad y seguridad.
Antes de ejecutar acciones destructivas, pide confirmación.
Cuando programes, prueba los cambios y explica qué verificaste.
No inventes resultados de herramientas.
```

Ejemplo de AGENTS.md

Instrucciones de un proyecto

```
# AGENTS.md

Proyecto: aplicación web.

- Usa pnpm.
- Ejecuta tests antes de dar una tarea por terminada.
- No edites archivos generados.
- La API vive en /apps/api.
- El frontend vive en /apps/web.
- No cambies el esquema de base de datos sin migración.
```

Consejo: pon en SOUL.md lo que debe acompañar a Hermes en todos tus proyectos; pon en AGENTS.md lo que solo tiene sentido dentro de un repositorio.

8. Cómo Hermes aprende y crea sus propias skills

La característica diferencial que más interesa en un agente de este tipo es el bucle de aprendizaje. Hermes no «reentrena» automáticamente el modelo base cada vez que hablas con él. Su mecanismo de mejora es principalmente procedural: conserva memoria y convierte procedimientos útiles en skills reutilizables.

El ciclo puede entenderse así:

Fase	Qué ocurre
1. Ejecutar	Hermes intenta resolver una tarea usando herramientas, memoria y skills.
2. Observar	Detecta qué pasos funcionan, qué falló y qué procedimiento puede repetirse.
3. Persistir	Guarda información estable en memoria cuando corresponde.
4. Abstraer	Cuando un procedimiento es reutilizable, puede convertirse en una skill.
5. Reutilizar	En futuras tareas, Hermes descubre la skill y carga su contenido solo cuando es pertinente.
6. Mejorar	Una skill existente puede actualizarse con mejores procedimientos a medida que se usa.

Las skills son memoria procedural

Una skill no es simplemente un prompt enorme que se envía siempre. Hermes mantiene descripciones breves y carga el contenido completo de una skill bajo demanda cuando la tarea lo requiere. Esto evita llenar el contexto de todas las instrucciones posibles en cada petición.

Las skills suelen vivir bajo HERMES_HOME/skills/ y contienen un SKILL.md. Pueden incluir scripts, referencias y plantillas.

Ejemplo de workflow que merece convertirse en skill

- «Cada lunes revisa mis repos, detecta PRs bloqueados, resume riesgos y crea un informe.»
- «Cuando te pase un paper, extrae metodología, resultados y limitaciones y genera una tabla comparativa.»
- «Para desplegar este proyecto, ejecuta los mismos pasos, verifica health checks y documenta el rollback.»

Cómo ayudar al aprendizaje

- Después de una tarea compleja, pide: «convierte este procedimiento en una skill reutilizable».
- Si una skill falla, pide que la revise incorporando el fallo y la solución.
- Mantén las skills modulares: una skill por workflow, no una skill gigantesca para todo.
- Usa scripts auxiliares para pasos deterministas; deja al modelo la parte de decisión.
- Revisa periódicamente las skills creadas automáticamente para evitar procedimientos obsoletos.

Importante: «se auto-mejora» significa que aprende procedimientos, contexto y preferencias mediante persistencia y skills; no significa que esté modificando los pesos del LLM subyacente.

9. Bots: crear agentes especializados

Bot Mode convierte los perfiles de Hermes en una colección de agentes especializados. Cada Bot puede tener su propio chat, rol, modelo, memoria, skills, credenciales y rutinas.

Crear un Bot desde Desktop

- Abre la pestaña Bots.
- Selecciona New Agent.
- Define Name, Title y Description.
- En Advanced puedes clonar otro perfil o empezar desde cero.
- Puedes fijar un modelo/proveedor específico para ese Bot.
- Define un SOUL.md personalizado para su identidad.
- Activa solo las skills, toolsets y servidores MCP necesarios.
- Decide si utilizará credenciales compartidas o separadas según el caso.

Ejemplos de Bots útiles

Bot	Modelo / capacidades	Objetivo
Coder	Modelo fuerte en código + terminal + git	Desarrollo, tests, refactors y PRs.
Researcher	Modelo fuerte en web + navegador	Investigación, fuentes y síntesis.
Content	Modelo rápido + skills de escritura	Borradores, repurposing y calendarios.
Ops	Modelo fiable + cron + terminal	Monitorización y tareas recurrentes.
Personal	Modelo general + memoria	Agenda, archivos y tareas personales.

Bot-to-Bot

La documentación actual permite que los Bots se mencionen y trabajen juntos en chats de grupo. También pueden ejecutar rutinas recurrentes y pasarse trabajo. Esto permite una arquitectura de «equipo de agentes» sin tener que convertir cada conversación en un único agente gigantesco.

Diseño recomendado

Un agente generalista para tareas ambiguas + Bots especialistas para workflows repetibles. Mantén cada Bot con el mínimo de herramientas que necesita.

10. Skills, plugins y MCP

Hermes tiene tres capas diferentes de extensibilidad. Entenderlas evita usar una herramienta demasiado compleja para un problema sencillo.

Mecanismo	Úsalo para	Ejemplo
Skills	Enseñar procedimientos y workflows.	Publicar un paquete, investigar un paper, hacer un release.
Plugins	Añadir o modificar capacidades de Hermes.	Herramientas, hooks, temas e integraciones.
MCP	Conectar servidores de herramientas externos.	GitHub, bases de datos, filesystem, APIs internas.

MCP

Model Context Protocol permite que Hermes descubra herramientas de servidores externos y las registre como herramientas del agente. Hermes admite servidores locales por stdio y servidores HTTP remotos, además de filtrado por servidor.

Ejemplo conceptual de MCP en config.yaml

```
mcp_servers:
  filesystem:
    command: "npx"
    args:
      - "-y"
      - "@modelcontextprotocol/server-filesystem"
      - "C:/Users/TU_USUARIO/Projects"
```

En Desktop puedes gestionar MCP desde Settings. Para un servidor nuevo, limita el alcance de filesystem a la carpeta que realmente necesita.

Plugins

La arquitectura de plugins permite añadir herramientas, hooks, temas e integraciones sin tocar el núcleo de Hermes. Los plugins de terceros deben considerarse código con privilegios: instala solo fuentes en las que confíes.

Herramientas integradas

El registro actual incluye web search/extract, navegador, terminal, archivos, memoria, búsqueda de sesiones, cron, delegación a subagentes, ejecución de código, generación de imágenes, TTS, visión, computer use y otras integraciones. La disponibilidad concreta depende del sistema, credenciales y toolsets habilitados.

11. Mensajería: hablar con Hermes desde tus apps

El gateway permite que el mismo agente que tienes en Desktop responda desde canales de mensajería. La documentación actual incluye Telegram, Discord, Slack, WhatsApp, Signal, SMS, Email, Matrix, Teams, LINE, Mattermost, Feishu/Lark, WeCom, Webhooks y más.

La ventaja es arquitectónica: no tienes «un bot de Telegram» separado y «un Hermes de escritorio» separado. El gateway puede conectar las plataformas al mismo ecosistema de perfiles, herramientas, memoria y cron.

Flujo recomendado

- Configura y prueba Hermes localmente primero.
- Crea un Bot específico si el canal va a ejecutar tareas sensibles.
- Conecta el proveedor de mensajería desde Settings / Gateway.
- Restringe quién puede activar el agente.
- Define qué toolsets tiene ese canal.
- Prueba primero consultas de solo lectura.
- Después habilita acciones con escritura, terminal o navegador.

Advertencia

Dar acceso a un agente desde WhatsApp/Telegram no convierte el canal en una interfaz inocua: si el Bot tiene terminal, filesystem, navegador o computer-use, un mensaje puede desencadenar acciones reales. Diseña el acceso como si estuvieras exponiendo una API de automatización.

Documentación oficial de Messaging Gateway
https://hermes-agent.nousresearch.com/docs/user-guide/messaging/

12. Cron: automatizar tareas recurrentes

Cron permite programar tareas one-shot o recurrentes y asociarlas con skills. Los resultados pueden entregarse al chat de origen, a archivos locales o a plataformas configuradas.

Ejemplos

Horario	Tarea
08:00 diario	Revisar agenda y generar briefing.
09:00 lunes	Revisar repos y resumir PRs pendientes.
18:00 diario	Resumir noticias de una lista de fuentes.
Viernes	Generar informe semanal de proyectos.
Cada 30 min	Comprobar una fuente/API y avisar solo si cambia algo.

Un cron puede cargar una o varias skills antes de ejecutar el prompt. También existe un modo sin agente en el que se ejecuta un script programado sin participación del LLM, útil cuando quieres determinismo y coste cero de inferencia.

Ejemplo

Concepto de cron con skill

```
cronjob(
  action="create",
  skill="blogwatcher",
  prompt="Comprueba novedades y genera un resumen.",
  schedule="0 9 * * *",
  name="Resumen diario"
)
```

Otra opción potente es asignar un modelo específico a cada job. Así un informe rutinario puede utilizar un modelo barato, mientras una tarea crítica puede fijar un modelo más potente.

Truco: evita crear jobs que «hagan todo». Divide la automatización en tareas pequeñas: recolectar → validar → sintetizar → entregar. Es más fácil depurar y más barato.

13. Computer Use, navegador, terminal y archivos

Aquí Hermes deja de parecerse a un simple chat. El agente puede interactuar con el ordenador, el navegador y el filesystem mediante herramientas.

Computer Use

Hermes puede conducir el escritorio en Windows, macOS y Linux: clicar, escribir, desplazarse y arrastrar. La implementación utiliza un driver de computer-use y puede trabajar en segundo plano sin mover necesariamente el cursor visible ni robar el foco de tu trabajo.

Esto permite workflows como: abrir una aplicación, revisar una interfaz, rellenar un formulario, comprobar un resultado y guardar el artefacto. Requiere especial cuidado porque el agente está actuando sobre tu ordenador real.

Computer Use oficial
https://hermes-agent.nousresearch.com/docs/user-guide/features/computer-use

Browser automation

Hermes dispone de automatización de navegador local y opciones cloud. Puede navegar, clicar, escribir, rellenar formularios y extraer información. También puede conectarse a Chrome/Brave/Chromium/Edge mediante CDP.

Terminal y archivos

- Ejecutar comandos.
- Leer, editar y parchear archivos.
- Ejecutar procesos en background.
- Buscar dentro de proyectos.
- Usar git y herramientas de desarrollo.
- Delegar subtareas a otros agentes.

Sandboxing

Cuando una tarea no necesita acceso directo al sistema, considera un backend aislado como Docker. La documentación de Hermes contempla distintos backends de terminal; el backend local es potente pero también el más sensible.

14. Configuración avanzada para exprimir Hermes

Una configuración potente no consiste en activar absolutamente todo. Consiste en asignar el modelo y las herramientas correctas a cada tipo de trabajo.

1. Usa perfiles

Los perfiles aíslan configuración, sesiones, skills y memoria. Esto es ideal para separar «Personal», «Trabajo», «Coding», «Research» o un cliente concreto.

2. Usa modelos auxiliares

No gastes el modelo principal en cada microtarea. Configura modelos auxiliares para compresión, visión, extracción web u otras tareas donde un modelo rápido sea suficiente.

3. Fallbacks

Configura fallback cuando una tarea sea importante. Si el proveedor principal falla, Hermes puede intentar una ruta alternativa. Prueba los fallbacks antes de depender de ellos en producción.

4. Contexto grande

Mantén ≥64K de contexto. Para agentes, el contexto no es solo la conversación: también entran instrucciones, resultados de herramientas, archivos y estado de trabajo.

5. Herramientas por perfil/Bot

Activa terminal, browser, computer-use y MCP solo donde sean necesarios. Menos herramientas también puede significar menos posibilidades de que el modelo elija una acción innecesaria.

6. Session search

Hermes puede buscar en sesiones anteriores almacenadas localmente. Esto es diferente de MEMORY.md: permite recuperar conversaciones concretas sin convertirlas todas en memoria permanente.

7. Quick Entry

Configura el atajo global de Quick Entry para lanzar instrucciones rápidas desde cualquier aplicación. Es especialmente útil para tareas como «resume este texto», «crea una nota», «revisa este archivo» o «ejecuta mi workflow X».

8. Artifacts

Usa la galería de artifacts para localizar rápidamente archivos, imágenes y enlaces producidos por tus sesiones.

9. No mezcles homes simultáneamente

La documentación advierte que no debes ejecutar dos procesos de agente escribiendo sobre el mismo HERMES_HOME al mismo tiempo, porque pueden pisarse cambios de memoria/estado.

15. Seguridad y buenas prácticas

Hermes puede tener acceso real al sistema. Por tanto, el riesgo no está solo en el modelo: está en la combinación modelo + herramientas + credenciales + datos + superficie de acceso.

Riesgo	Mitigación
API key filtrada	Guardar secretos en .env/gestor de credenciales; nunca en markdown ni repos.
Agente con terminal total	Usar toolsets mínimos o backend aislado cuando sea posible.
MCP demasiado amplio	Limitar rutas, servidores y herramientas expuestas.
Bot accesible desde mensajería	Restringir usuarios/canales y usar un perfil específico.
Computer Use	Evitar tareas irreversibles sin confirmación humana.
Exportar un perfil	Revisar el archivo antes de compartirlo: puede incluir memoria, sesiones y USER.md.
Dos agentes con el mismo home	Usar perfiles/home distintos.

Backups

Haz backup de tu configuración, SOUL, skills, memoria y proyectos, pero no publiques credenciales. La documentación de distribución/exportación indica que los archivos de credenciales como auth.json y .env se excluyen de determinados exports, pero el contenido de memoria y sesiones puede incluir información personal.

Prompt injection

Cuando Hermes navega por la web o procesa archivos externos, puede encontrar instrucciones maliciosas dirigidas al agente. No trates el contenido de una web, email o README externo como autoridad equivalente a SOUL.md o a tus instrucciones del proyecto. Para tareas de alto riesgo, separa lectura de ejecución y exige confirmación antes de acciones destructivas.

16. Setup recomendado: Hermes potenciado al máximo

Esta es una arquitectura práctica para un usuario avanzado que quiere potencia sin convertir la instalación en un proyecto infinito.

Nivel 1 — base

- Windows 10/11 + Hermes Desktop.
- OpenRouter como proveedor flexible.
- Modelo principal ≥64K con buen tool calling.
- SOUL.md bien definido.
- USER.md y MEMORY.md revisados periódicamente.

Nivel 2 — productividad

- Skills para workflows repetibles.
- Quick Entry.
- Session search.
- Artifacts.
- 2–3 Bots especializados.

Nivel 3 — automatización

- Cron para tareas recurrentes.
- Mensajería solo para Bots concretos.
- Modelo barato para jobs rutinarios.
- Fallback para trabajos importantes.

Nivel 4 — integración

- MCP para GitHub / bases de datos / herramientas internas.
- Browser automation.
- Computer Use cuando sea realmente necesario.
- Backend Docker para tareas que no necesiten tocar el host.

Nivel 5 — híbrido

- OpenRouter para tareas complejas.
- Modelo local para tareas privadas o económicas.
- Bots que pinneen modelos diferentes.
- Skills compartidas y memoria separada por perfil.

Ejemplo de ecosistema

Componente	Configuración
Hermes Desktop	Windows local

Componente	Configuración
Cerebro	OpenRouter + modelo fuerte
Modelo barato	OpenRouter free/low-cost para auxiliares y jobs
Modelo privado	Ollama/LM Studio para tareas locales
Coder Bot	Terminal + filesystem + git
Research Bot	Web + browser + skills de investigación
Ops Bot	Cron + terminal + alertas
MCP	Solo servidores necesarios
Messaging	Canal restringido a Bots concretos

Prompts que sacan partido al agente

1) Convertir un procedimiento en skill

```
Analiza cómo resolviste esta tarea. Extrae un procedimiento
repetible y conviértelo en una skill llamada <nombre>.
Incluye precondiciones, pasos, validaciones, errores comunes
y cuándo NO debe utilizarse.
```

2) Mejorar una skill existente

```
Usa la skill <nombre>. Si encuentras un fallo o un paso
innecesariamente frágil, corrígela para que el workflow
sea más robusto. Conserva compatibilidad con los casos
que ya funcionan.
```

3) Crear un Bot especialista

```
Crea un Bot especializado en <objetivo>. Dale una identidad
clara, un modelo adecuado, solo las herramientas necesarias
y una skill inicial para su workflow principal. Define también
qué acciones requieren confirmación humana.
```

El principio general es simple: primero consigue fiabilidad, después automatiza y finalmente especializa. Un agente que hace diez cosas mediocremente es menos útil que tres Bots pequeños que hacen una cosa muy bien.

17. Fuentes oficiales

La guía se ha construido priorizando documentación oficial de Hermes Agent / Nous Research y documentación oficial de OpenRouter. Los enlaces siguientes son los puntos de referencia para comprobar cambios de versión, modelos disponibles y comportamiento actual.

Hermes Desktop — página oficial
https://hermes-agent.nousresearch.com/desktop
Hermes — documentación
https://hermes-agent.nousresearch.com/docs/
Instalación
https://hermes-agent.nousresearch.com/docs/getting-started/installation
Quickstart
https://hermes-agent.nousresearch.com/docs/getting-started/quickstart
Windows nativo
https://hermes-agent.nousresearch.com/docs/user-guide/windows-native
Desktop
https://hermes-agent.nousresearch.com/docs/user-guide/desktop
Qué hace cada archivo
https://hermes-agent.nousresearch.com/docs/user-guide/which-file-does-what
Memoria persistente
https://hermes-agent.nousresearch.com/docs/user-guide/features/memory
Personality / SOUL.md
https://hermes-agent.nousresearch.com/docs/user-guide/features/personality
Bot Mode
https://hermes-agent.nousresearch.com/docs/user-guide/bot-mode
Skills / creación
https://hermes-agent.nousresearch.com/docs/developer-guide/creating-skills
Tools & Toolsets
https://hermes-agent.nousresearch.com/docs/user-guide/features/tools
MCP
https://hermes-agent.nousresearch.com/docs/user-guide/features/mcp
Messaging Gateway
https://hermes-agent.nousresearch.com/docs/user-guide/messaging/
Cron
https://hermes-agent.nousresearch.com/docs/user-guide/features/cron
Browser automation
https://hermes-agent.nousresearch.com/docs/user-guide/features/browser
Computer Use
https://hermes-agent.nousresearch.com/docs/user-guide/features/computer-use
Configuración de modelos
https://hermes-agent.nousresearch.com/docs/user-guide/configuring-models
OpenRouter — crear API key
https://openrouter.ai/docs/api/api-reference/api-keys/create-keys
OpenRouter — Free Models
https://openrouter.ai/collections/free-models
OpenRouter — Free Models Router
https://openrouter.ai/docs/cookbook/get-started/free-models-router-playground

Nota de actualidad: los modelos gratuitos, nombres de modelos, límites y algunos detalles de la interfaz cambian con frecuencia. Antes de publicar un tutorial, conviene volver a comprobar el catálogo y el instalador oficial.

Checklist final de instalación

- ☐ Hermes Desktop instalado desde la fuente oficial.
- ☐ Primer chat probado correctamente.
- ☐ OpenRouter conectado o modelo local operativo.
- ☐ Modelo con ≥64K de contexto.
- ☐ SOUL.md definido.
- ☐ USER.md / MEMORY.md revisados.
- ☐ AGENTS.md creado para cada proyecto importante.

- ☐ Skills instaladas y probadas.
- ☐ Bots separados para tareas con necesidades distintas.
- ☐ MCP limitado a servidores necesarios.
- ☐ Cron probado con una tarea inocua.
- ☐ Mensajería restringida.
- ☐ Computer Use activado solo si aporta valor.
- ☐ API keys y credenciales fuera de repositorios y documentos compartidos.

Fin de la guía